

考试科目名称 编译原理; A 卷

考试方式: 闭卷 考试日期: 2022年01月04日 教师: 魏恒峰

院系 (专业): 软件学院 (软件工程) 年级: 2019级

姓名: _____ 学号: _____ 成绩: _____

题号	一	二	三	四	五	六	总分			
分数										

注意事项:

- 诚信考试, 不得作弊。
- 若对题意有疑问, 请及时提出。
- 题目不是按照难度排列的, 请注意合理分配时间。
- 为了避免连锁错误造成的影响, 请尽量给出关键步骤。
- 题目中的分值仅表明各小题之间的相对权重, 不代表实际分数。
- 字迹要尽可能工整, 解题过程要尽可能清晰。

题目 1 (正则表达式与自动机 (15 = 7 + 8 分))

考虑正则表达式 $r = (a \mid b)^*a^+$ 。

- (1) 请使用 Thompson 构造法构造等价的 NFA。
- (2) 请使用子集构造法构造等价的 DFA。(注: 请标明状态之间的对应关系)

题目 2 (LL 语法分析 (15 = 2 + 5 + 6 + 2 分))

考虑文法 G :

$$D \rightarrow TL \quad (1)$$

$$T \rightarrow \text{int} \mid \text{real} \quad (2)$$

$$L \rightarrow L, \text{id} \mid \text{id} \quad (3)$$

- (1) 请消除文法 G 中的左递归 (记新文法为 G');
- (2) 请为文法 G' 计算必要的 FIRST 集合与 FOLLOW 集合;
- (3) 请为文法 G' 构造预测分析表;
- (4) 请问文法 G' 是否是 $LL(1)$ 文法, 并说明理由。

题目 3 (ANTLR4 与“优先级上升算法” (15 分))

考虑 ANTLR4 改造后的如下 `expr` 文法 (上层规则为 `stat : expr[0] ';' ;`):

```

expr[int _p]
: (
  | ID
)
( {4 >= $_p}? '*' expr[5]
  | {3 >= $_p}? '+' expr[4]
)*
;

```

请给出 $1 + 2 * 3 + 4 * 5$ 在该文法下对应的语法分析树。请给出关键的解释, 点到即可, 不必面面俱到。

题目 4 (**LR**语法分析 (20 = 6 + 10 + 4 分))

考虑文法 G (已作增广处理):

$$S' \rightarrow S \quad (0)$$

$$S \rightarrow (S)S \quad (1)$$

$$S \rightarrow (L)S \quad (2)$$

$$S \rightarrow \epsilon \quad (3)$$

$$L \rightarrow \mathbf{id} \quad (4)$$

$$L \rightarrow L, \mathbf{id} \quad (5)$$

(1) 请为文法 G 计算必要的 FIRST 集合与 FOLLOW 集合;

(2) 请为文法 G 构造 $LR(1)$ 自动机;

注意: 为了尽量统一状态编号, 便于批改, 当计算CLOSURE时, 请按照文法编号大小顺序加入新项。

当计算GOTO(I, X)时, 请按照 I 中项的出现顺序依次考虑可能的转移符号 X 。

要求: 请说明归约的设置条件。

(3) 请为文法 G 设计 $LR(1)$ 分析表; 该文法是 $LR(1)$ 文法吗? 请说明理由。

题目 5 (语法制导定义与翻译 (15 = 6 + 9 分))

考虑如下文法 G ,

$$P \rightarrow D$$

$$D \rightarrow D ; D$$

$$D \rightarrow T \text{ id}$$

$$D \rightarrow \text{func id } \{D\}$$

$$T \rightarrow \text{int}$$

请设计语法制导的翻译方案, 完成下列任务。你需要自行定义合适的属性。

- (1) 打印程序 P 一共声明了多少个 **id** (包括 $T \text{ id}$ 与 **func id**)。

例如, 句子 `int id ; func id {int id}` 中共声明了 3 个 **id**。

- (2) 打印程序 P 中每个变量 **id** (不考虑函数名 **id**) (在 $\{\}$ 中) 的嵌套深度。

例如, 在句子 `int id ; func id {int id}` 中, 第一个变量 **id** 的嵌套深度为 0, 第二个变量 **id** 的嵌套深度为 1。

题目 6 (中间代码生成 (20 分))

考虑如下计算 a, b 最大公约数的程序:

```
1: procedure GCD( $a, b$ )  
2:   while  $a \neq b$  do  
3:     if  $a > b$  then  
4:        $a = a - b$   
5:     else  
6:        $b = b - a$   
7:   return  $a$ 
```

请基于布尔表达式与控制流语句回填翻译方案 (参考图1、图2、图3与图4) 为 **while** 循环 (第 2-6 行) 生成中间代码。

要求: 请使用图示(如语法树等)展示产生式与相应规则的使用情况。

产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow assign$	$S.code = assign.code$
$S \rightarrow if (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow if (B) S_1 else S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' S.next)$ $\quad \parallel label(B.false) \parallel S_2.code$
$S \rightarrow while (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

图 1: 控制流语句的语法制导定义

产生式	语义规则
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\quad \parallel gen('if' E_1.addr \text{ rel } op E_2.addr 'goto' B.true)$ $\quad \parallel gen('goto' B.false)$
$B \rightarrow true$	$B.code = gen('goto' B.true)$
$B \rightarrow false$	$B.code = gen('goto' B.false)$

图 2: 布尔表达式的语法制导定义

1) $B \rightarrow B_1 \parallel M B_2$	{ $backpatch(B_1.falselist, M.instr);$ $B.truelist = merge(B_1.truelist, B_2.truelist);$ $B.falselist = B_2.falselist;$ }
2) $B \rightarrow B_1 \&\& M B_2$	{ $backpatch(B_1.truelist, M.instr);$ $B.truelist = B_2.truelist;$ $B.falselist = merge(B_1.falselist, B_2.falselist);$ }
3) $B \rightarrow ! B_1$	{ $B.truelist = B_1.falselist;$ $B.falselist = B_1.truelist;$ }
4) $B \rightarrow (B_1)$	{ $B.truelist = B_1.truelist;$ $B.falselist = B_1.falselist;$ }
5) $B \rightarrow E_1 \text{ rel } E_2$	{ $B.truelist = makelist(nextinstr);$ $B.falselist = makelist(nextinstr + 1);$ $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto -');$ $gen('goto -');$ }
6) $B \rightarrow \text{true}$	{ $B.truelist = makelist(nextinstr);$ $gen('goto -');$ }
7) $B \rightarrow \text{false}$	{ $B.falselist = makelist(nextinstr);$ $gen('goto -');$ }
8) $M \rightarrow \epsilon$	{ $M.instr = nextinstr;$ }

图 3: 布尔表达式的回填翻译方案

1) $S \rightarrow \text{if}(B) M S_1$	{ $backpatch(B.truelist, M.instr);$ $S.nextlist = merge(B.falselist, S_1.nextlist);$ }
2) $S \rightarrow \text{if}(B) M_1 S_1 N \text{ else } M_2 S_2$	{ $backpatch(B.truelist, M_1.instr);$ $backpatch(B.falselist, M_2.instr);$ $temp = merge(S_1.nextlist, N.nextlist);$ $S.nextlist = merge(temp, S_2.nextlist);$ }
3) $S \rightarrow \text{while } M_1 (B) M_2 S_1$	{ $backpatch(S_1.nextlist, M_1.instr);$ $backpatch(B.truelist, M_2.instr);$ $S.nextlist = B.falselist;$ $gen('goto' M_1.instr);$ }
4) $S \rightarrow \{ L \}$	{ $S.nextlist = L.nextlist;$ }
5) $S \rightarrow A ;$	{ $S.nextlist = \text{null};$ }
6) $M \rightarrow \epsilon$	{ $M.instr = nextinstr;$ }
7) $N \rightarrow \epsilon$	{ $N.nextlist = makelist(nextinstr);$ $gen('goto -');$ }
8) $L \rightarrow L_1 M S$	{ $backpatch(L_1.nextlist, M.instr);$ $L.nextlist = S.nextlist;$ }
9) $L \rightarrow S$	{ $L.nextlist = S.nextlist;$ }

图 4: 控制流语句的回填翻译方案